

Reviewer Pack — Minimal Local Induction Dimension and SAT Clause Subset Entr...

Entry 51d97bb0be7c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 04:00:07 UTC

Minimal Local Induction Dimension and SAT Clause Subset Entropy Correlation

Entry ID: 51d97bb0be7c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 04:00:07 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Induction Dimension) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

For every d -regular graph G , the minimal local induction dimension ($\text{ltd}(G)$) of its associated SAT formula φ_G is linearly correlated with its clause subset entropy $E(\varphi_G)$, such that $\text{ltd}(G) = \Theta(E(\varphi_G))$.

Rationale (proposer's reasoning):

The connection between algebraic topology and computational complexity can reveal novel insights into proof complexity. Local induction dimension (LID) measures the complexity of a space, which might correlate with the structure of SAT formulas, potentially providing a new way to understand clause subset entropy.

Taxonomy category: Algebraic_Topology (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d6c7d5e29da9d7c8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all d -regular graphs G with $n \leq 40$, the correlation coefficient r between $\text{ltd}(G)$ and $E(\varphi_G)$ is ≥ 0.5 across at least 24 of the 30 seeds, and the p -value of the linear regression model is ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_d_regular_graph(n, d):
    if n % d != 0:
        raise ValueError("Graph size must be a multiple of the degree")

    graph = {i: [] for i in range(n)}
    degree_count = [0] * n

    for node in range(n):
        available_neighbors = [i for i in range(n) if i != node and len(graph[i]) < d]
        neighbors_to_add = random.sample(available_neighbors, min(d - degree_count[node], len(available_neighbors)))

        for neighbor in neighbors_to_add:
            graph[node].append(neighbor)
            graph[neighbor].append(node)
            degree_count[node] += 1
            degree_count[neighbor] += 1

    return graph

def compute_local_induction_dimension(graph):
    n = len(graph)
    adjacency_matrix = [[0 for _ in range(n)] for _ in range(n)]

    for node, neighbors in graph.items():
        for neighbor in neighbors:
            adjacency_matrix[node][neighbor] = 1
            adjacency_matrix[neighbor][node] = 1

    # Gaussian elimination to find the rank of the adjacency matrix
    rank = 0
    for i in range(n):
        if adjacency_matrix[i][i] == 0:
            found_pivot = False
```

```

        for j in range(i + 1, n):
            if adjacency_matrix[j][i] != 0:
                for k in range(n):
                    adjacency_matrix[i][k], adjacency_matrix[j][k] = adjacency_matrix[j][k], adjacency_matrix[i][k]
                    found_pivot = True
                break
            if not found_pivot:
                continue

        rank += 1
        denom = Fraction(adjacency_matrix[i][i])
        for j in range(n):
            adjacency_matrix[i][j] /= denom

        for j in range(n):
            if i != j:
                factor = Fraction(adjacency_matrix[j][i])
                for k in range(n):
                    adjacency_matrix[j][k] -= factor * adjacency_matrix[i][k]

    return rank

def compute_clause_subset_entropy(clauses):
    n = len(clauses)
    total_clauses = sum(len(c) for c in clauses)
    entropy = 0
    for clause in clauses:
        p = Fraction(1, total_clauses)
        entropy -= p * math.log2(p)

    return entropy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = generate_random_d_regular_graph(n, 3)
        clauses = [] # Placeholder for SAT formula generation
        for node in range(n):
            clause = [node + 1] * (len(graph[node]) - 1) # Simplified clause representation
            clauses.append(clause)

        ltd = compute_local_induction_dimension(graph)
        entropy = compute_clause_subset_entropy(clauses)

        results.append({
            "n": n,
            "ltd": ltd,
            "entropy": entropy
        })

    correlation_coefficient = 0
    for i in range(len(results)):
        for j in range(i + 1, len(results)):

```

```

        correlation_coefficient += (results[i]["ltd"] - results[0]["ltd"]) * (results[j]["entr
correlation_coefficient /= (len(results) * (len(results) - 1))

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(r["n"] for r in results),
    "conjecture_holds": abs(correlation_coefficient) >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ecd53494.py", line 126, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ecd53494.py", line 89, in run_trial
    graph = generate_random_d_regular_graph(n, 3)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ecd53494.py", line 20, in generate_random_d_regular_graph
    raise ValueError("Graph size must be a multiple of the degree")

```

ValueError: Graph size must be a multiple of the degree

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient and p-value as required by the support condition. | next: Review the code for potential errors that cause the crash and ensure the test can run to completion. If the error is resolved, rerun the test with a larger sample size of d-regular graphs to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13794
2	propose	ollama_remote	glm4:latest	0	0	14523
3	preregistration	ollama_remote	glm4:latest	0	0	9714
4	novelty	ollama_remote	glm4:latest	0	0	8940
5	novelty	ollama_remote	glm4:latest	0	0	9026
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24358
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14151
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14411
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14725
10	judge	ollama_remote	glm4:latest	0	0	12410

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 136052 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/51d97bb0be7c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/51d97bb0be7c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/51d97bb0be7c.tar.gz` (if generated)